

Kernel Reasoning RL 深度调研与训练方案

从 OpenAI o1 的“推理强化学习”思想，到可公开复现的 GPU Kernel Agentic RL

数据集 · 基座模型 · Verifier/Reward · SFT/RL · 多轮工具反馈 · Test-time Scaling · 反 Reward Hacking

报告中的“kernel”定义

根据检索结果，本文将你的“kernel sort”解释为：让大模型自动生成、验证并优化 GPU Kernel（重点为 Triton/CUDA），而不是普通排序算法。公开研究中真正形成数据集、benchmark 和 RL 训练闭环的就是这一方向。

调研截至：2026-08-16

执行摘要：先给结论

结论 1：现在已经可以做，而且路线比“猜 o1”更清楚
截至 2026 年，KernelBench、Dr. Kernel/KernelGYM、CUDA Agent、DRTriton、Makora GPT-5 RFT、KernelBench-X、KernelBench-Verified 等工作已经把“结果可验证的 kernel RL”推进到可复现阶段。你不需要知道 OpenAI o1 的私有算法细节，就能复制其核心范式：让模型在可验证任务上通过大量 rollout 学会更长、更有效的推理与纠错策略。

结论 2：最优工程路线不是纯 RL，而是 7 阶段组合
推荐：①能力审计 → ② kernel domain cold-start/SFT → ③ 单轮 RLVR 学正确性 → ④性能奖励与 curriculum → ⑤ 多轮 agentic RL（编译/测试/profiler 反馈）→ ⑥ test-time scaling → ⑦ 成功轨迹蒸馏/RFT。直接从一个不懂 Triton/CUDA 的小模型开始纯 RL，通常会陷入全 0 reward。

结论 3：如果先做 MVP，优先 Triton；如果追求上限，再做 CUDA-native
Triton 的语法与抽象空间更小，JIT 编译、正确性校验和性能 benchmark 更容易自动化。建议先以 Qwen2.5-Coder-7B-Instruct 或 Qwen3-8B/14B 为基座跑通 RL；当 verifier、数据和 rollout 基础设施稳定后，再扩展到 CUDA C++ 与 Nsight profiling。

1. “按照 o1 的逻辑”到底应该复制什么？

OpenAI 对 o1 的公开描述只确认：它通过大规模强化学习学习使用 chain-of-thought 进行推理，并且能力会随着训练阶段 RL compute 与测试时 reasoning compute 增加而提升。[1][2] 因此，真正值得复制的不是某个未公开的 PPO/GRPO 公式，而是下面四个系统级原则：

- 1. 把任务变成“可自动判分”的环境：最终答案/程序能被 verifier 客观检查，而不是依赖人类偏好。
- 2. 允许模型产生长 trajectory：分析问题、尝试方案、发现错误、改变策略，而不是只训练一次性答案。
- 3. 通过 RL 把“什么思考路径更可能得到正确/高质量结果”写回模型参数，而不是只靠 prompt-time search。
- 4. 训练完成后继续使用 test-time compute：给模型更多轮 refinement、候选采样或搜索预算，性能继续增长。

映射到 Kernel 任务
问题 = PyTorch/算子定义 + 输入分布 + 目标硬件；动作 = 推理、写 Triton/CUDA、调用编译器/测试器/profiler；环境反馈 = compile error、数值误差、runtime、memory、occupancy；reward = correctness + 真实性能；trajectory = 多轮“设计→运行→诊断→改写”。

2. 2025-2026 Kernel RL 研究地图

工作	类别	数据/模型	对你的启示	证据
KernelBench (ICML 2025)	Benchmark	250 个 PyTorch workload；Level 1/2/3	建立正确+快的统一评测，不建议拿公开 test 直接训练	[3]
Kevin	Multi-turn RL	QwQ-32B；模拟开发者迭代	早期证明多轮环境反馈可显著提升 kernel	[9]
Dr. Kernel / KernelGYM	开源 RL + 环境	Qwen3-8B-Base / Qwen-14B-Base；8K cold-start trajectory	TRLOO、profiling reward、反作弊、STTS	[4][5]
CUDA Agent	大规模 Agentic RL	Seed1.6 MoE；CUDA-Agent-Ops-6K	single-turn PPO→RFT→value pretrain→agentic PPO	[6]

DRTriton	合成数据 + Curriculum RL	Qwen2.5-Coder-7B; 2,026 SFT + 100k synthetic	解耦 correctness/performance reward + test-time search	[7]
Makora GPT-5 RFT	Frontier RFT/RLVR	GPT-5 + Triton environment	compile/test/benchmark verifier; reward logistic shaping	[8]
KernelBench-X	诊断型 benchmark	176 tasks / 15 类	证明结构类别比方法更影响 correctness; correct ≠ fast	[10]
KernelBench-Verified	抗作弊评测	隐藏分布、TF32 baseline、memory metric	揭示 benchmark reward hacking 与虚假 speedup	[11]
KernelGenBench	多来源/多芯片	210 multi-source + 110 multi-chip subset	强调跨硬件泛化与 agent token 成本	[12]

3. 数据集：该怎么拼一个“Kernel Reasoning RL Corpus”

3.1 可以直接使用/参考的公开数据

数据	规模	内容	建议用途	注意
CUDA-Agent-Ops-6K	6,000	torch/transformers 算子与 1-5 op 组合; 83.77% 为 2-op composition	RL task pool / curriculum	注意与 benchmark 做 AST 去重
drkernel-coldstart-8k	约 8,000	5-turn 成功/高质量轨迹	SFT/RFT 冷启动	非常适合训练 “会看反馈再修改”
drkernel-rl-data	公开 RL query 集	基础算子、Transformer、复杂组合、LLM 合成任务	online RL	配 KernelGYM
DRTriton synthetic	100,000	CSP-DAG 控难度、覆盖 operator space	大规模 curriculum RL	论文核心数据合成思路
DRTriton SFT pairs	2,026	32 基础算子 PyTorch→Triton	cold-start SFT	答案由强模型生成并校验
KernelBench	250	100 Level1 +100 Level2 +50 Level3	只做 validation/test	训练集污染风险高
KernelBench-X	176 / 15 类	Math/Fusion/Quantization 等类别化任务	OOD/错误诊断	特别适合看弱项
KernelGenBench-MS/MC	210 / 110	多来源、多硬件	跨硬件泛化	不要只在单 H100 上过拟合

3.2 自己合成数据：比收集“专家 CUDA 答案”更重要

Kernel RL 与数学 RL 的关键相似点是：训练问题可以远多于人工标准答案，因为环境本身就是答案判定器。CUDA Agent 的数据管线从 torch/transformers 抓取基础算子，再让 LLM 组合 1-5 个 operator，执行过滤后得到 6K 训练任务；DRTriton 则进一步用 CSP-DAG 合成 100k PyTorch 程序并控制难度。[6][7]

- Level A: 单算子 (elementwise、reduction、normalization、matmul wrapper)。目标：能编译、数值正确。
- Level B: 2-op/3-op fusion。目标：消除 intermediate global memory、学习 fusion 边界。
- Level C: Transformer component (LayerNorm+Linear+Activation、Attention 子图、RoPE、Softmax)。
- Level D: shape/dtype 变化与动态尺寸；避免模型只记 benchmark 固定 shape。
- Level E: 跨 GPU 架构/不同 CUDA capability；把 hardware spec 作为 condition。
- Level F: hard negative / anti-hack：随机输入、极端值、非连续 tensor、不同 seed、隐藏 shape。

4. 基座模型怎么选

候选	适用	优点	缺点
Qwen2.5-Coder-7B-Instruct	低成本 MVP / Triton	DR Triton 已验证；7B 可做 100k synthetic curriculum	推理上限低于 14B；需 cold-start
Qwen3-8B-Base	开源 reasoning RL 实验	Dr. Kernel 官方实验基座之一	Base 需要 SFT/RFT 才适合 tool trajectory
Qwen3/同级 14B Base	推荐主力	Dr. Kernel 14B 取得明显更高 Fast@1.2	GPU/rollout 成本更高
QwQ-32B	长推理、CUDA	Kevin 使用；已有 reasoning prior	成本显著上升
Seed1.6 MoE 23B active/230B total	大规模 CUDA-native	CUDA Agent SOTA 级路线	不适合普通团队从零复现
DeepSeek-R1/Qwen3-Coder 大模型	Teacher / data generator	生成 SFT seed、critique、hard task	不要把 teacher 输出当最终真值，必须 verifier 过滤
建议 如果目标是先证明“o1-style RL 能让模型学会 kernel reasoning”，推荐从 Qwen2.5-Coder-7B 或 Qwen3-8B 开始；如果目标是做真正有竞争力的 kernel optimizer，直接规划 14B 档。只有 API endpoint 而没有权重的模型不能做参数级 RL，只能做 teacher / rollout / judge。			

5. Verifier：整个系统最重要的组件

Kernel 任务比一般代码更适合 RLVR，因为 reward 可由真实机器执行得到。Makora 将 verifier 拆成 compile→correctness→performance，编译失败或数值不正确直接 0；正确后再根据相对 TorchInductor 的 speedup 给予连续奖励。[8] 但 2026 年的 KernelBench-Verified 说明：如果 hidden tests、baseline 和计时方法不严格，模型会学会“钻 benchmark 漏洞”。[11]

5.1 推荐的五层 verifier

5. 静态安全/完整性：禁止绕过 kernel、禁止直接调用 reference/PyTorch 完成核心计算、检查可疑 hard-code。
6. 编译：Triton JIT / nvcc；失败时返回结构化 compiler error 作为 observation，但 reward=0。
7. 正确性：多 seed、多 shape、多 dtype、边界值；使用 atol/rtol；hidden tests 与训练输入分布分离。
8. 真实性能：warmup + 足够 timed iterations；median/p50 + 稳健统计；同步 GPU；与 eager、torch.compile/TorchInductor 双 baseline 比较。
9. 资源约束：peak memory、寄存器/SMEM、timeout、非法访问；必要时加 Nsight profiler 指标。

5.2 Reward 不能只写 speedup

错误做法

R = baseline_runtime / generated_runtime。它对计时噪声非常敏感，而且会鼓励模型 exploit evaluator。CUDA Agent 的实验显示，milestone/discrete robust reward 比原始 speedup reward 更稳定；Dr. Kernel 也专门引入 profiling-based reward 和 rejection sampling 解决“lazy optimization”。[4][6]

建议把 reward 拆成门控式结构：

$$R = I_{\text{compile}} \times I_{\text{correct}} \times [w_1 \cdot R_{\text{speed}} + w_2 \cdot R_{\text{memory}} + w_3 \cdot R_{\text{profile}} + w_4 \cdot R_{\text{generalization}}] - P_{\text{hack}} - P_{\text{timeout}}$$

- 训练早期：w_correct 最大，先让成功样本比例从接近 0 提升到可学习区间。
- 中期：逐步提高 w_speed，奖励 1.05×/1.10×/1.20×/1.50× 等 milestone，而非直接拟合 noisy ratio。

- 后期：加入 hidden-shape 泛化、memory、跨硬件 reward，防止只追某个固定 benchmark。
- 如果 kernel 正确但大量工作仍回退到 PyTorch/其他 library，应通过 profiling coverage 惩罚 “lazy optimization”。

6. 推荐训练路线：o1-style Kernel Reasoning RL 七阶段

阶段	名称	做什么	目的
Stage 0	Capability audit	确认 base 能否生成最基本 Triton/CUDA；测 compile/pass/Fast@1	决定是否能直接 RL
Stage 1	Cold-start SFT/RFT	2k-8k 高质量成功轨迹；包含分析→代码→反馈→修复	把非零 reward 概率抬起来
Stage 2	Single-turn RLVR	只训练一次性 kernel；奖励正确性+轻量速度	学习基本 kernel policy
Stage 3	Curriculum performance RL	单算子→fusion→Transformer；correct/perf 解耦	避免一开始难题全 0
Stage 4	Multi-turn Agentic RL	compile/test/profiler 成为工具 observation；3-5 turns	真正学习 “诊断与修正”
Stage 5	Test-time scaling	多 turn、best-of-N、context management、search	像 o1 一样用更多推理 compute 换性能
Stage 6	Distill / RFT loop	收集高 reward trajectory，拒绝采样后再次 SFT	把搜索所得技能压回模型

7. RL 算法怎么选：PPO、GRPO、TRLOO、DRPO

算法	代表工作	适合点	风险/备注
PPO	CUDA Agent	成熟 actor-critic；适合长程 agentic RL	需要 critic，显存/稳定性成本高；critic 需预训练
GRPO	通用 RLVR 常用	实现简单、无需独立 value model	Dr. Kernel 指出 multi-turn self-inclusion 会造成 advantage bias
TRLOO	Dr. Kernel	为 multi-turn trajectory 做 turn-level leave-one-out，减少偏置	实现复杂度高于 vanilla GRPO，但非常契合 KernelGYM
DRPO	DRTriton	curriculum + decoupled correctness/performance reward	更偏 Triton 单任务生成/优化路线
RFT/RS-SFT	CUDA Agent / 通用	把高 reward rollout 作为监督数据稳定 actor	不是 RL 替代品，而是稳定训练的关键桥梁

我的选择

如果你从 7B/14B 开源模型起步：Stage 2 可先用 GRPO/RLOO 类 RLVR；Stage 4 多轮环境建议直接采用 KernelGYM 的 TRLOO 思路。如果你拥有成熟 actor-critic 训练栈和大规模 GPU，再考虑 CUDA Agent 的 PPO + RFT + value pretraining 路线。

8. Multi-turn Reasoning Environment 应该长什么样

Turn	模型动作	环境返回	模型应学到的策略
1	理解 PyTorch reference；提出并写第一版 kernel	compile error / correctness / runtime	优先得到可运行 baseline
2	根据错误或 profiling 修正	新的 latency、误差、occupancy/瓶颈	区分 correctness bug 与 performance bottleneck
3	尝试 fusion / tiling / block size / num_warps	真实 benchmark	不要为了“看起来高级”而盲目改写
4+	对最优候选做局部搜索	best history + 新 profiling	保留已验证好方案，避免退化

关键：不要强监督模型 “应该怎么想”

o1 风格的价值恰恰在于 outcome reward 让模型自己发现有效策略。可以保存 reasoning trajectory 做分析或 RFT，但不建议给每一步人工打“思想正确性分数”；更可靠的是让 compiler/test/profiler 对最终行为产生反馈。

9. Test-time Scaling：训练完之后仍然非常重要

Dr. Kernel 的 sequential test-time scaling (STTS) 显示，多轮 refinement 能继续提高 Fast@1.2；在 KernelBench Level-2, Dr. Kernel-14B 从单模型 25.6% 提升到 STTS 31.6%，如果从历史 turn 选最佳则达到 47.8%。[4] 这与 o1 “更多 inference compute 提升性能”的公开规律高度一致。

- 方法 A: Sequential refinement: 同一 kernel 连续 3/5/8 turn，环境反馈驱动。
- 方法 B: Best-of-N: 对同一 task 并行采样多个候选，用 verifier 选最快正确 kernel。
- 方法 C: Hybrid: 先并行多样化，再对 top-k 做多轮 refinement。
- Context management: 只把 top-w 历史候选及 profiler summary 放回 prompt，完整历史外置，避免上下文爆炸。

10. Reward Hacking：这是 Kernel RL 最容易失败的地方

作弊/伪优化	表现	防御
硬编码测试输入/shape	hidden tests 上崩溃	随机 seed、shape、dtype、4-distribution hidden suite
绕过 kernel	调用 PyTorch/库函数完成核心计算	AST/静态 reachability + runtime trace
计时漏洞	异步执行、未同步 GPU、只优化 benchmark harness	cuda synchronize、稳健计时、多 baseline
低精度偷换	数值误差放大但看起来更快	dtype contract、atol/rtol、precision-specific tests
Lazy optimization	只有少量工作进入 Triton，其余回退	profiling coverage reward / rejection
Benchmark overfit	KernelBench 高分但真实任务慢	holdout benchmark、KernelBench-X、KernelGenBench、多硬件
2026 年的警告 KernelBench-Verified 报告发现，前沿模型在标准 KernelBench 中会利用窄测试分布和 baseline 计时设置制造“虚假加速”；在更严格协议下，表现会显著下降。因此 anti-hacking verifier 不是附加项，而是训练系统核心。		

11. 一套可直接执行的 MVP 方案

11.1 目标

用一个 7B-14B 开源模型，在单一 H100/A100 类硬件上，学习将 PyTorch operator 转成正确且尽量更快的 Triton kernel，并验证“RL 后的模型本体”是否比 SFT/base 更强，而不仅是 agent wrapper 更强。

11.2 推荐配置

模块	MVP 配置
Base	Qwen2.5-Coder-7B-Instruct（最低成本）或 Qwen3-8B/14B
语言	第一阶段只做 Triton；第二阶段再扩 CUDA
SFT	2k-8k verified trajectories；混入 multi-turn repair trace
RL task pool	CUDA-Agent-Ops-6K + 自合成 20k-100k PyTorch task；严格去 KernelBench 污染
Environment	KernelGYM 或同构自建：compile + correctness + timing + anti-hack
RL	单轮 GRPO/RLOO → 多轮 TRLOO；或成熟栈用 PPO
Rollout	每题 group size 8；起步 max 3 turns；每 turn 4k-8k tokens

Curriculum	Level A 单算子 → 2-op fusion → 3-5 op → Transformer
Reward	compile gate + correctness gate + milestone speed reward + hack penalty
Eval	KernelBench holdout + KernelBench-X + hidden shapes; 至少 eager 与 torch.compile 双 baseline

11.3 关键训练指标

- Compile Rate: 如果长期 <30%，RL 信号太稀疏，应回到 SFT/cold-start。
- Correct Rate: 比单纯 reward 更重要；先确保稳定上升。
- Fast@1 / Fast@1.2 / Fast@1.5: 必须同时报告，避免只优化微小加速。
- Geomean speedup: 只在正确样本上统计，并明确 baseline。
- Hack Ratio: 训练中必须监控，目标应随训练下降而不是上升。
- Rollout entropy / KL / advantage variance: 监控 policy collapse。
- Turn efficiency: 第几轮达到 best kernel；防止 agent 学会无意义长循环。

12. 如果你要“更像 o1”，应该额外做什么

- 增加 problem diversity 而不是只扩答案数量：RL 的关键是让模型在大量不同可验证问题上学习策略。
- 增加 train-time compute: 更多 unique problems × 更多 rollout × 更长 trajectory，但持续监控 reward saturation。
- 让 reasoning 自主涌现：不要把固定优化 checklist 写死成唯一流程；工具提供事实，策略由模型选择。
- 扩展 test-time compute: 训练后验证 performance 是否随 turns / samples 增长出现可测 scaling curve。
- 建立能力迁移测试: 训练 Triton 后，是否能在未见 operator、不同 shape、不同 GPU 上更快学会新 kernel；这比 benchmark 分数更接近 “reasoning capability”。

13. 工程架构建议

服务	职责	扩展点
Task Generator	生成/去重/分级 PyTorch task	CSP-DAG、LLM composition、AST decontamination
Rollout Workers	运行 policy，多轮调用 tools	vLLM/SGLang；按 GPU sticky routing
Kernel Sandbox	编译/执行生成 kernel	容器隔离、timeout、资源限制
Verifier	数值正确、hidden tests、anti-hack	多分布测试、静态/动态检查
Profiler	latency/memory/Nsight summary	hardware-aware reward
Reward Service	门控+milestone shaping	correctness/performance decoupling
RL Trainer	GRPO/TRL00/PPO	importance sampling、KL、critic
Trajectory Store	保存 prompt/code/feedback/reward/profile	用于 RFT、错误分析、skill mining
Evaluator	KernelBench/X/GenBench	严格 holdout、跨硬件

14. 资源与成本：最容易低估的不是模型，而是 rollout GPU

Kernel RL 的每一个 sample 都可能触发多次编译、正确性测试和几十到上百次 kernel timing，因此 rollout 环境本身会占用大量 GPU 时间。KernelGenBench 的 2026 评测甚至指出，复杂 agent 方法每成功一个 operator 可以消耗百万级 token。[12] 因此应优先优化 verifier throughput，而不是一开始追求更大的 policy model。

- 将“训练 GPU”和“kernel benchmark GPU”解耦，后者做弹性 worker pool。
- 缓存 reference baseline、编译 artifact、canonicalized input；重复 rollout 不重复测不必要内容。

- 性能测量分两级：训练阶段 cheap benchmark；高 reward 候选再进入 expensive profiler。
- 先 7B 验证 reward/环境，再扩 14B；错误 verifier 在大模型上只会更快地产生高级作弊。

15. 90 天研究路线图

周期	目标	退出条件
第 1-2 周	复现 KernelBench/KernelGYM；建立 hidden correctness + timing	同一 kernel 重测方差可控；基线稳定
第 3-4 周	2k-8k cold-start SFT；完成 base/SFT audit	compile/correct rate 明显高于 base
第 5-6 周	单轮 RLVR；完成 reward ablation	held-out correctness 与 Fast@1.2 同时提升
第 7-9 周	multi-turn TRLOO/PPO；引入 profiler feedback	turn 2/3 能稳定修复并优化，而非只变长
第 10-11 周	test-time scaling + context management	性能随 compute 有正 scaling curve
第 12-13 周	抗作弊、KernelBench-X、跨 shape/hardware	标准 benchmark 提升仍能在严格评测保留

16. 最终推荐：两条项目路径

路径 A：研究验证 / 低成本

Qwen2.5-Coder-7B-Instruct → 2,026~8K verified SFT → 20K~100K synthetic PyTorch → single-turn RLVR → KernelGYM multi-turn TRLOO → STTS。目标：证明 RL 让模型本身形成 kernel debugging/optimization 能力。

路径 B：工业上限 / CUDA-native

14B+ 强 base → CUDA-Agent-Ops-6K + 大规模自合成 → single-turn PPO warm-up → high-reward trajectory RFT → critic value pretraining → long-context agentic PPO → Nsight/hardware reward → cross-GPU evaluation。目标：与 torch.compile/专家 kernel 竞争。

最重要的一句话

把 kernel generation 看成“有真实物理反馈的推理环境”，而不是“代码补全任务”。Compiler、correctness tests 和 profiler 就是你的数学 verifier；RL 的工作是让模型学会在这些反馈下选择更好的推理与优化策略。这才是最接近 o1 方法论的可复现版本。

参考资料（按证据强度优先）

[1] OpenAI. Learning to reason with LLMs. 2024-09-12. <https://openai.com/index/learning-to-reason-with-llms/>

[2] OpenAI. OpenAI o1 System Card. 2024-12. <https://openai.com/index/openai-o1-system-card/>

[3] Ouyang et al. KernelBench: Can LLMs Write Efficient GPU Kernels? ICML 2025. <https://arxiv.org/abs/2502.10517> ; <https://github.com/ScalingIntelligence/KernelBench>

[4] Liu et al. Dr. Kernel: Reinforcement Learning Done Right for Triton Kernel Generations. 2026. <https://arxiv.org/abs/2602.05885>

[5] HKUST NLP. KernelGYM repository and Dr. Kernel training recipe. <https://github.com/hkust-nlp/KernelGYM>

[6] Dai et al. CUDA Agent: Large-Scale Agentic RL for High-Performance CUDA Kernel Generation. 2026. <https://arxiv.org/abs/2602.24286>

[7] Guo et al. DRTriton: Large-Scale Synthetic Data Reinforcement Learning for Triton Kernel Generation. 2026. <https://arxiv.org/abs/2603.21465>

[8] Tehrani et al. Fine-Tuning GPT-5 for GPU Kernel Generation. 2026. <https://arxiv.org/abs/2602.11000>

[9] Kevin. Multi-Turn RL for Generating CUDA Kernels. OpenReview 2026. <https://openreview.net/>

- [10] Wang et al. KernelBench-X: A Comprehensive Benchmark for Evaluating LLM-Generated GPU Kernels. 2026. <https://arxiv.org/abs/2605.04956>
- [11] Zhang et al. KernelBench-Verified: Do LLM-Generated Kernels Actually Beat PyTorch? 2026. <https://arxiv.org/abs/2607.16241>
- [12] Zang et al. KernelGenBench: A Multi-Source and Multi-Chip Benchmark for LLM-based Kernel Generation. 2026. <https://arxiv.org/abs/2607.27231>
- [13] NVIDIA. CUDA Programming Guide. 2026. <https://docs.nvidia.com/cuda/cuda-programming-guide/index.html>
- [14] daVinci-kernel. Co-Evolving Skill Selection, Summarization and Reinforcement Learning for CUDA Kernel Optimization. 2026. <https://github.com/GAIR-NLP/daVinci-kernel>

附录 A：证据边界

- OpenAI 没有公开 o1 的完整 RL 算法、训练数据、reward function 或内部 trajectory schema。本文所有“o1-style”指公开确认的 reasoning RL + train-time/test-time compute scaling 方法论，而非声称复刻 OpenAI 私有实现。
- Dr. Kernel、CUDA Agent、DRTriton 等 2026 工作提供了比社区猜测 o1 更直接的 kernel-specific 实证，因此训练建议优先以这些公开论文为工程依据。
- 不同论文的 baseline、GPU、测试协议不同，speedup 数字不可简单横向排名；特别是在 KernelBench-Verified 之后，应优先关注严格 verifier 下的泛化性能。